

Audit Pine — Technique & Business

Projet : Pine 🌲 — « The Ansible control plane that doesn't need a control plane »

Date : 2 juillet 2026

Méthode : audit multi-agents (6 agents parallèles : architecture, cœur métier, sécurité/runner, performance, frontend/UX, business) sur le code réel.

Périmètre : ~13 400 LOC Go, 5 700 LOC JS (web), TUI Bubble Tea, CLI, API REST. Build/vet/tests **verts** au moment de l'audit.

1. Verdict en une page

Pine est un projet **remarquablement mûr pour ~1 mois de développement solo**. L'architecture est propre et acyclique, les moteurs de scan et de plan sont écrits comme des fonctions pures bien testées, et l'idée produit — « *terraform plan pour Ansible* » — est un vrai différenciateur défendable sur une catégorie que personne n'occupe.

Mais l'outil **exécute Ansible sur des serveurs de production via une API REST sans aucune authentification**, et un endpoint fuite des secrets d'inventaire en clair. Ce sont les deux verrous absolus avant toute mise en production ou toute démarche commerciale.

Axe	Note	Synthèse
Architecture	🟢 Solide	Couches acycliques, moteurs purs testés, single-binary. Dette sur le store JSON et la concurrence.
Cœur métier (scan/plan)	🟡 Bon	Logique tri-valuée rigoureuse, mais 1 fuite de secret + écarts de précedence Ansible.
Sécurité	🔴 Critique	Pas d'auth sur l'API , RCE via URL git, fuite vault. Bloquant.
Performance	🟡 Correct	Jamais profilé sur gros repos ; I/O non optimisées (scan mono-thread, store full-rewrite).
Frontend/UX	🟢 Solide	Vanilla JS discipliné, XSS maîtrisé, design cohérent. Monolithe non testé.
Business	🟢 Potentiel réel	Catégorie inoccupée, angle CI fort. Gaps entreprise (RBAC/SSO) et bus factor solo.

Top 5 des priorités absolues

- 🔴 **Authentifier l'API REST** + bind `127.0.0.1` par défaut (§4.1)
- 🔴 **Rédiger `Redact()` l'endpoint `/lineage`** — fuite de secrets en clair (§3.1)
- 🔴 **Allowlister les URL/schémas git** — RCE via transport `ext` (§4.2)
- 🟡 **Borner la concurrence des jobs** (worker pool) + réconcilier les jobs « running » au boot (§2)
- 🟡 **Indexer/paginer le store JSON** avant qu'il ne plafonne (§2, §5)

2. Architecture

Synthèse agent : architecture en couches propre et acyclique, dépendances externes minimales, moteurs de scan et de plan en fonctions pures bien testées. Les principales dettes concernent le store JSON, la concurrence des

Points forts

- **Dépendances acycliques bien orientées** : `model` en feuille, `server` seul point de câblage ; dépendances externes minimales (Bubble Tea, Lipgloss, yaml.v3) → single-binary autonome.
- **Moteurs `plan.Compute` et `scanner.Scan` en fonctions pures** sans état, fortement testés.
- **Abstraction `tui.Engine`** satisfaite à la fois par le manager en-process et par le client HTTP → même TUI en local ou attachée à un daemon.
- **Écritures atomiques** (`tmp` + `rename`), garde-fous path traversal, modèle tri-valué `run/skip/unknown` dans le modèle de données.

Dettes structurelles

Sévérité	Constat	Localisation	Correctif
High	Store mono-processus, aucun verrou inter-processus : le mutex ne protège que la mémoire ; <code>serve / tui / local / plan</code> ouvrent chacun le même répertoire → course sur <code>state.json / pipelineruns.json</code> .	<code>store.go:28-31</code> ; <code>cmd/pine/main.go:132,201</code>	Verrou exclusif (flock) au démarrage.
High	Persistance linéaire sans cache ni index : <code>ListJobs</code> relit tous les jobs à chaque appel, <code>GetPipelineRun</code> charge tous les runs, chaque <code>Save</code> réécrit le fichier entier ; <code>pipelineruns.json</code> croît sans borne.	<code>store.go:182-205</code> ; <code>collections.go:170-313</code>	Cache/index en mémoire, pagination, purge.
High	Aucun plafond de concurrence des jobs : <code>StartJob</code> lance une goroutine + ansible sans sémaphore ; le scheduler (30 s) peut déclencher toutes les schedules dues d'un coup.	<code>runner/jobs.go:118-163</code> ; <code>scheduler.go:146</code>	Worker pool borné avec file.
High	Erreurs d'écriture ignorées, pas de réconciliation : ~82 erreurs ignorées ; le save terminal ignore l'erreur et l'état de run vit en mémoire → un crash laisse le job running à jamais .	<code>runner/jobs.go:225,252</code> ; <code>store.go:35</code>	Remonter les erreurs ; marquer les jobs non terminaux au démarrage.
Medium	Logique de plan divergente selon le point d'entrée : <code>computePlan</code> (web) injecte <code>TaskDurations</code> , <code>HostFacts</code> et le vault ; <code>localEngine.Plan</code> et <code>cmdPlan</code> non → plans différents selon web/TUI/CLI. L' <code>Engine</code> ne couvre que 8 des 40 endpoints.	<code>tui/engine.go:45-53</code> ; <code>server.go:486-488</code> ; <code>main.go:609</code>	Extraire un <code>Manager.Plan</code> partagé.
Medium	Cache scan mutable non invalidé (<code>Scan()</code> retourne le pointeur partagé), VaultPassword en clair dans <code>state.json</code> , store peu testable (pas d'interface <code>Store</code> , <code>Job</code> sans champ <code>Error</code> , <code>Scan/Compute</code> sans <code>context</code>).	<code>manager.go:116-143</code> ; <code>model.go:180-186</code>	Copie + TTL, secrets isolés, interface <code>Store</code> , champ <code>Error</code> , <code>context</code> .

3. Cœur métier — scanner & plan (bugs)

Synthèse agent : le cœur métier est soigné et fidèle à l'esprit « honest engine » (logique tri-valuée cohérente, propagation des variables manquantes, gardes anti-cycles, redaction réutilisable). Mais un vrai problème de sécurité et des écarts de précedence Ansible entament ponctuellement la promesse d'honnêteté.

Points forts

- **Logique tri-valuée rigoureuse** : `triAnd/triOr/triNot` implémentent correctement Kleene à 3 valeurs ; `in` /comparaisons propagent l'undefined avec la liste des variables manquantes.
- **Gardes anti-cycle solides** : `seen` + `maxImportDepth` sur les `import_tasks`, fixpoint borné sur les dépendances de rôles, cap à 10 itérations sur `expandNestedVars`.
- **Redaction de secrets réutilisable** partagée entre hygiène, lineage et resolve (avec exclusion des faux positifs : `server_tokens`, toggles on/off).
- **Déchiffrement vault transient et prudent** : mot de passe en fichier temporaire supprimé, masquage des blobs non déchiffrés.

Bugs identifiés

3.1 CRITIQUE — Fuite de secrets : l'endpoint `/lineage` renvoie les valeurs non redactées

`server.go:709` appelle `plan.Lineage()` puis `writeJSON` sans jamais appeler `out.Redact()`. Or `Lineage()` remplit `VarLineage.Value/Chain` avec les valeurs brutes des `group_vars` / `host_vars` / `role_vars/defaults` — mots de passe en clair et blobs `$ANSIBLE_VAULT` inclus. Tous les autres consommateurs redactent (l'endpoint `resolve` dont la docstring promet « *Secrets are redacted before the values ever leave Pine* », et la CLI). Le type `LineageResult` a même une méthode `Redact()` prévue pour ça, mais le handler l'oublie. Un simple `GET /api/repos/{id}/lineage?inventory=..&host=..` divulgue les secrets d'inventaire en JSON.

Loc : `internal/server/server.go:703-715` (vs 737). **Fix** : ajouter `out.Redact()` avant `writeJSON` ; idéalement faire porter la redaction par `Lineage()` elle-même (défense en profondeur).

3.2 Précédence inversée : play vars écrase vars_files (contraire à Ansible)

Dans `resolve.go:158-163` les couches sont ajoutées `vars_file` puis `play_vars`, et `add()` fait gagner la dernière écriture. Pour une même clé, Pine retient play vars ; Ansible fait l'inverse (play vars niveau 12 < vars_files niveau 14). Même ordre inversé dans le moteur estimé (`vars.go:163-167`). Les deux moteurs sont cohérents entre eux mais **tous deux faux vis-à-vis d'Ansible**. Non couvert par les tests.

Fix : inverser l'ordre + test de précedence clé-définie-dans-les-deux.

3.3 Le moteur de plan ignore les role vars/main.yml

`varResolver.effective()` ne merge que `r.Defaults`, jamais `r.Vars` (`vars/main.yml`). Conséquence : une condition `when:` ou un `{{ }}` référénçant une variable de role `vars/main.yml` est évalué sans elle → **faux unknown** + variable listée manquante, alors qu'Ansible la connaît (précédence 15). Incohérent avec `resolve.go` / `lineage.go` qui, eux, ajoutent la couche `role_vars`.

Loc : `plan.go:308-313`. **Fix** : merger aussi `r.Vars` après `play vars/vars_files`.

3.4 Détection de référence de rôle par sous-chaîne → faux positifs hygiène/impact

`hygiene.go:77` et `impact.go:297` détectent l'usage d'un rôle via `strings.Contains(t.Args, role.Name)`. Un rôle `db` est « référencé » par toute tâche contenant `mariadb` ; un rôle `name` est toujours « référencé » (les args d' `include_role` commencent par `name:`). Effets : hygiène rate des rôles inutilisés, **blast radius surestimé**. Le modèle expose pourtant `Task.RoleRef` (nom exact).

Fix : utiliser `t.RoleRef` (égalité stricte).

3.5 🟡 Patterns d'hôte ! (exclusion) et & (intersection) silencieusement ignorés

`MatchHosts` saute les termes `! / &`. Pour `webserver:!staging`, Pine renvoie **tous** les webserver, staging inclus, et leur attribue des verdicts `run/skip` fermes — **sans marqueur d'incertitude**, ce qui contredit la promesse « honest engine ».

Fix : marquer ces hôtes `unknown` a minima ; idéalement implémenter exclusion/intersection.

3.6 🟢 Autres (low/info)

- **Égalité type-laxiste** (`cmpEq` via `fmt.Sprintf("%v")`): `bool true == string "true"`, `int 1 == "1"` → verdict potentiellement erroné sans incertitude signalée. `eval.go:580-588`.
- **unused_vars faux négatif** quand une var est définie en 2 endroits ; secrets < 6 caractères non détectés. `hygiene.go:434,319`.
- **Plan estimé jamais rédigé** : un secret vault déchiffré peut apparaître dans un nom/arg de tâche (déclenché par un mot de passe fourni volontairement, mais aucune défense en profondeur). `plan.go:347-355`.
- **serial en % ou en liste non géré** (`atoiSafe` → 0 → un seul batch, silencieusement). `plan.go:299,605`.

4. Sécurité (runner / server)

Synthèse agent : API REST sans auth exécutant Ansible en production — faille dominante qui amplifie toutes les autres.

Points forts

- Redaction `resolve / lineage / redactRepo`, exécution en `argv` **sans shell**, vault jamais loggué, fichiers de secrets temporaires en `0600`.

Faibles

4.1 🔴 CRITIQUE — Aucune authentification/autorisation sur l'API REST

`New()` ne monte aucun middleware d'auth. `POST /api/jobs` exécute `ansible-playbook`, `POST /api/repos` clone du git — **sans identifiant**. Le port `8743` bind sur **toutes les interfaces**. Cette faille amplifie toutes les autres.

Loc : `server.go:54-112` ; `cmd/pine/main.go:284`. **Fix :** auth obligatoire devant `/api/`, bind `127.0.0.1` par défaut.

4.2 🔴 CRITIQUE — RCE via URL git + lecture de fichiers arbitraires

`addRepo` accepte une URL arbitraire vers `git clone` (`manager.go:81`) ; le transport `ext` lance un shell → **RCE sans auth**. `addRepo` accepte aussi un chemin absolu quelconque, et `repoFile` suit les symlinks via `os.ReadFile`.

Fix : allowlist des schémas git + `GIT_ALLOW_PROTOCOL`, allowlist de chemins, `EvalSymlinks`.

4.3 🟡 High — Fuite du vault password (sync non rédigé, 0644) + CSRF

`syncRepo` renvoie le `Repo` brut **sans redactRepo** (`server.go:356`) → VaultPassword en clair à chaque sync. `state.json` en `0644` (lisible localement). Aucune vérif `Origin / Host` → un site malveillant peut faire `POST /api/jobs` (CSRF).

Fix : `redactRepo` au retour de sync, permissions `0600`, valider `Host` et `Origin`.

4.4 🟡 Medium — Playbook non confiné, plan sans redaction, DoS et courses

`job.Playbook` positionnel non confiné (préfixe `-` vu comme option). `computePlan` renvoie les vault déchiffrés sans `Redact()`. `StartJob` spawn illimité ; `jobEvents` lit tout le log en mémoire sans limite SSE. `advancePipeline` fait lecture-modif-`Save` sans verrou. Secrets temporaires dans `TMPDIR` partagé.

Fix : chemins relatifs + séparateur `--`, rédiger le plan, worker pool + limite SSE, verrou par entité.

5. Performance & optimisation

Synthèse agent : Pine n'a jamais été profilé pour des gros repos type debops (240 playbooks / 203 rôles). Les points chauds ne sont pas dans l'algorithmique du plan mais dans les **I/O** et quelques **$O(n^2)$ évitables**.

Points forts

Cache de scan en mémoire par repoID, regex en variables de package (pas de recompilation), écritures atomiques, streaming de logs propre (fan-out bufferisé avec drop des lents), bornes anti-explosion (`maxImportDepth` ...), ETag/304 sur les assets.

Optimisations prioritisées

Sévérité	Point chaud	Localisation	Gain attendu / correctif
High	Scan reparse tout le repo en YAML, séquentiel mono-thread, à chaque sync — 3 parcours d'arbre + <code>yaml.v3</code> sur des milliers de fichiers sur un seul cœur, aucun cache par mtime.	<code>scanner.go:24–28</code> ; <code>playbook.go:98</code> ; <code>role.go:67</code>	Fusionner en 1 parcours, paralléliser (errgroup + sémaphore), cache incrémental par <code>(path, mtime, size)</code> . Debops : plusieurs s → < 1 s.
High	ListJobs relit tout jobs/ à chaque appel , sans pagination — appelé par <code>/api/jobs</code> (liste entière) ET <code>/api/stats</code> , pollés toutes les 10 s. Des milliers de syscalls par poll sous le RLock global.	<code>store.go:182–205</code> ; <code>server.go:153,518</code>	Index jobs en mémoire, ? <code>limit=&offset=</code> , compteur <code>running</code> en mémoire pour <code>/stats</code> .
High	Scan à froid dans le handler HTTP, sans singleflight — le 1 ^{er} <code>/scan</code> après boot bloque le client ; N requêtes concurrentes lancent N scans complets. Cache perdu au redémarrage.	<code>manager.go:124–144</code>	<code>singleflight</code> par <code>repoID</code> , réchauffement au boot, persistance <code>scan.<id>.json</code> .
High	Magic var groups reconstruite par hôte → O(hôtes²) — <code>eff["groups"]</code> (tous groupes × tous hôtes) réalloué H fois alors qu'il est host-indépendant.	<code>vars.go:133–141</code> ; <code>plan.go:340–356</code>	Calculer <code>allGroups</code> une fois par play, injecter la même référence ; lazy si <code>groups</code> non référencé.
High	Collections JSON réécrites en entier ; pipelineruns.json non borné et rechargé entièrement pour lire 1 run → O(N²) sur la durée de vie, jamais purgé.	<code>collections.go:179–215,272–313</code>	1 fichier par run (ou JSONL append-only), purge/plafond, lecture par ID directe.
Medium	Expression when re-tokenisée/re-parsée pour chaque hôte — l'AST ne dépend pas de l'hôte.	<code>plan.go:480–500</code> ; <code>eval.go:53,215</code>	Parser 1× par tâche (closure/AST), évaluer H fois ; mémoiser par chaîne.
Medium	Réponses API non compressées ; /scan renvoie tout le ScanResult (multi-Mo) à chaque appel.	<code>server.go:124–127,359–380</code>	Middleware gzip (5-10×), endpoint <code>/scan</code> « slim » + détail à la demande.
Medium	Statut du job relu du disque toutes les 2 s par client SSE connecté.	<code>server.go:619–644</code>	Publier les changements via le channel de logs ; servir le dernier <code>Job</code> en mémoire.
Medium	Web UI reconstruit le DOM par innerHTML sans virtualisation ni pagination.	<code>web/app.js</code> (multiples)	Pagination/filtrage, virtualisation (fenêtrage), rendu incrémental (<code>DocumentFragment</code> + <code>rAF</code>).
Low/Info	<code>tickSchedules</code> réécrit <code>schedules.json</code> par schedule et par tick ; <code>resolveImportTasks</code> re-parse les fichiers importés à chaque nœud ; <code>listRel</code> utilise <code>filepath.Walk</code> (<code>lstat</code>) au lieu de <code>WalkDir</code> .	—	Flush groupé, mémoisation du parsing, <code>WalkDir</code> .

6. Frontend & UX

Synthèse agent : vanilla JS monolithique (5 747 lignes) d'une qualité d'exécution nettement au-dessus de la moyenne « no-framework » : construction DOM disciplinée via `el()` + `TextNode` qui neutralise quasi tout le risque XSS malgré 69 usages d' `innerHTML` . Vraies faiblesses : maintenabilité (un seul fichier, aucun test JS) et accessibilité clavier.

Points forts

- **Surface XSS très maîtrisée** : `el()` passe toute donnée non fiable par `createTextNode` ; les données `repo/vars/logs` ansible ne touchent jamais `innerHTML` sans `esc()` (vérifié sur le highlighter YAML/INI et le rendu SSE live).
- **Cohérence design forte** : mêmes tokens `:root` (`--bg #0b0f0e` , `--accent #4ade80` , `--accent2 #22d3ee`) dans l'app, le site `website/` et la TUI (palette adaptative light+dark). Rare pour un projet solo.
- **Gestion erreurs/loading/empty de bon niveau** : wrapper `api()` unique, 42 empty states, 25 usages de `skeletonRows` , cycle de vie propre (`onCleanup` ferme l'EventSource et clear les intervals à chaque navigation).
- **Live logs SSE** avec reconnexion et bascule terminal → fetch complet.

À corriger

Sévérité	Constat	Localisation
 Medium	Monolithe de 5 747 lignes sans modules ni tests JS — état mutable dispersé (State + globales), aucun <code>package.json</code> / <code>*.test.js</code> /playwright. Principal risque de maintenabilité.	<code>web/app.js:1-5747</code>
 Medium	Éléments interactifs <code>span</code> / <code>div</code> non accessibles au clavier — onglets, en-têtes repliables, tokens de variable : ni <code>tabindex</code> , ni <code>role</code> , ni handler clavier.	<code>app.js:1156,2213,1328,1336</code>
 Low	Pas de focus-trap dans les modales ; <code>:focus-visible</code> absent (peu visible sur fond sombre) ; clé <code>html</code> de <code>el()</code> = footgun XSS latent (aucun bug vivant) ; pas de virtualisation des grandes listes.	<code>app.js:183-208,28</code> ; <code>style.css:190-197</code>
 Info	Parité web/TUI partielle : la web UI expose 14 sections, la TUI n'en couvre que 5 (parcourir + lancer). L'ambition « web+tui+cli+api » est inégale.	<code>tui/tui.go:78</code>

Recommandations : découper `app.js` en modules ES (servables sans build via `type=module`), quelques tests de fumée Playwright sur les parcours clés (run, plan, job log), convertir les contrôles cliquables en `<button>` , documenter que la web UI est la surface autoritaire.

7. Business — marché, cible, monétisation, marketing

L'unique idée défendable de Pine : « *terraform plan pour Ansible* » — un moteur d'analyse statique tri-valué qui raisonne `run/skip/unknown` avant l'exécution.

7.1 Marché & positionnement

Ansible reste massivement utilisé pour les VMs, le bare-metal et l'équipement réseau, là où Terraform ne va pas. Le paysage concurrentiel :

Concurrent	Forces	Ce que Pine a en plus
Red Hat AWX (gratuit, défaut)	Écosystème, notoriété	Léger (pas de k8s+Postgres+Redis), analyse statique
Ansible Automation Platform (payant)	RBAC, SSO, support entreprise	Le <i>plan</i> statique, le prix, la simplicité
Semaphore UI (jumeau le plus proche)	Auth, teams, LDAP	Analyse statique (Semaphore n'en a pas)
Rundeck / Spacelift / env0	Prouvent le modèle plan/policy-gate-in-CI	Spécifique Ansible

Verdict : occuper la catégorie inoccupée — « **analyse statique / plan-and-policy Ansible pour la CI** ».

7.2 Cible prioritaire

Équipes Platform/SRE avec de gros monorepos Ansible.

- *Pourquoi* : le blast radius et le plan tri-valué répondent à « qu'est-ce que ça va toucher avant que ça parte ? ».
- *Douleur* : `--check` s'exécute contre chaque hôte ; les reviewers approuvent les PR à l'aveugle.

7.3 Différenciateurs défendables

1. « **terraform plan pour Ansible** » — moteur statique tri-valué.
2. **Analyse honnête** — `unknown` + variables manquantes nommées (pas de faux-semblant).
3. **Blast radius comme CI gate** (`pine impact` , exit code 3).
4. **Single binary** — pas de k8s/Postgres/Redis.

7.4 Gaps produit qui bloquent une vente entreprise

-  **Aucune authentification** (cf. §4.1).
- **Pas de RBAC / SSO / audit log.**
- **Store JSON single-writer, pas de HA.**
- Pas de notifications, pas de gestionnaires de secrets externes (Vault, ASM...).
- **Mainteneur solo, pas d'entité de support** → bloque les deals entreprise (bus factor).

7.5 Monétisation

- **Open-core** : cœur OSS gratuit, features d'équipe payantes (RBAC/SSO/audit).
- **Pine Cloud pour la CI** : app hébergée qui commente les PR avec le plan + le blast radius.
- **Édition Team self-hosted** une fois le RBAC en place.
- **Support + conseil** (audit de parc Ansible).

7.6 Go-to-market & marketing

- **Coin d'entrée = la CI** : `pine plan / impact` en GitHub Action.
- Courtiser **Jeff Geerling** et la communauté Ansible.
- **Show HN** : « *terraform plan for Ansible* ».
- `r/ansible` , `r/selfhosted` ; SEO comparatif vs AWX/Semaphore.

Punchlines prêtes à l'emploi :

« terraform plan, for Ansible. » « Know what your playbook will touch before it touches production. » « No Kubernetes. No Postgres. No Redis. One binary. » « Blast radius for every Ansible PR. »

7.7 Risques business

- **Bus factor solo** — bloque les deals entreprise.
- **Red Hat possède le narratif AWX** (AAP).
- **Semaphore a déjà auth/LDAP** et pourrait ajouter l'analyse statique.
- **TAM Ansible mature / en déclin lent** dans les shops cloud-native.

8. Feuille de route recommandée

Sprint 0 — Sécurité (bloquant, avant tout usage réseau)

- Auth sur `/api/` (token/session) + bind `127.0.0.1` par défaut (§4.1)
- `Redact()` sur `/lineage` et `redactRepo` sur `syncRepo` (§3.1, §4.3)
- Allowlist schémas git + `GIT_ALLOW_PROTOCOL` + `EvalSymlinks` (§4.2)
- Validation `Host / Origin` (anti-CSRF), permissions `0600` sur `state.json`

Sprint 1 — Robustesse

- Worker pool borné pour les jobs + réconciliation des jobs `running` au boot (§2)
- Verrou inter-processus (flock) sur le store (§2)
- Corriger précedence `vars_files/play vars` et role `vars/main.yml` (§3.2, §3.3)
- `RoleRef` exact en hygiène/impact ; marquer `unknown` sur patterns `! / &` (§3.4, §3.5)

Sprint 2 — Scalabilité

- Scan parallèle + cache incrémental par mtime (§5)
- Index jobs + pagination `/api/jobs` ; `/stats` sans I/O (§5)
- `singleflight` + persistance du scan ; magic `groups` calculée 1x (§5)
- gzip + endpoint `/scan` slim (§5)

Sprint 3 — Go-to-market

- GitHub Action `pine plan / impact` (le coin CI)
- RBAC minimal (fondation open-core)
- Découpe `app.js` en modules + tests Playwright de fumée
- Show HN + articles comparatifs SEO

Annexe — Méthode

Audit conduit par 6 agents parallèles sur le code réel (Read/Grep/Glob/Bash), chacun spécialisé (architecture, cœur métier scanner/plan, sécurité runner/server, performance, frontend/UX, business), puis consolidé. Métriques de base : ~13 400 LOC Go, 21 fichiers de tests (`go test ./...` vert, `go vet` clean), 5 747 LOC `app.js` , 35

commits sur ~1 mois (9 juin → 1 juillet 2026), un seul contributeur. Toutes les sévérités et localisations `fichier:ligne` proviennent de l'inspection directe du code au commit `90d9de1` .